PROCESADORES DEL LENGUAJE

PRÁCTICA | LAVA

Analizador del lenguaje

Curso 2025-26

Campus de Colmenarejo

uc3m | Universidad Carlos III de Madrid

Tabla de contenidos

Introducción ... 3
Objetivos ... 3
Especificación del lenguaje ... 4
- Comentarios ... 5
- Palabras reservadas ... 5
- Variables y tipos ... 5
  - Número entero ... 5
  - Número real ... 6
  - Carácter ... 6
  - Booleano ... 6
- Operaciones ... 6
  - Aritméticas ... 6
  - Booleanas ... 6
  - Comparativas ... 6
- Reglas de precedencia ... 6
- Variables de tipo registro ... 7
- Control de flujo ... 9
  - Salto condicional ... 9
  - Bucle condicional ... 9
- Funciones ... 10
- Estructura del programa ... 11
- Sentencias ... 12
- Controles semánticos ... 13
  - Reglas de tipos ... 13
  - Conversiones automáticas ... 13

Diseño avanzado ... 14
- Generación de código intermedio ... 14
- Recuperación de errores ... 14

Entregas ... 15
- Analizador léxico - P1 ... 15
- Analizador sintáctico P2 ... 16
- Análisis semántico y partes opcionales - P3 ... 18

Introducción

La práctica de la asignatura de Procesadores del Lenguaje consistirá en la implementación de un compilador capaz de analizar programas de un lenguaje denominado Lava, que contiene expresiones aritmético-lógicas, registros, funciones y estructuras de control. También será capaz de comprobar la corrección en términos de los tipos de datos utilizados.

La práctica se desarrollará junto con los contenidos teóricos sobre los fundamentos del análisis léxico, sintáctico y semántico. Se espera que el alumno conozca la implementación completa de un analizador en sus fases y use estos conocimientos para detectar errores en el uso de tipos e informar al programador que usará el lenguaje Lava.

Objetivos

La práctica se divide en las siguientes partes o bloques de contenidos a realizar:

1. P1 Generación del **analizador léxico** o **lexer**
    1. Capaz de reconocer los elementos del lenguaje a nivel léxico.
    2. Implementación a código utilizando ply.lex
2. P2 Generación del **analizador sintáctico** o **parser**
    1. Diseño de la gramática, incluyendo símbolos terminales (tokens generados por el lexer), símbolos no terminales y reglas de producción, capaz de reconocer el lenguaje especificado.
    2. Implementación a código utilizando ply.yacc
3. P3 Ampliación del **parser** para completar el **análisis semántico**
    1. Registrar y tratar variables declaradas por el programador, almacenando el tipo con el que se declaran.
        1. Se deberá crear una estructura de tipo tabla para almacenar los símbolos (variables de tipo básico/nativo)
        2. Se deberá crear una estructura de tipo tabla para almacenar los registros (variables de tipo complejo/registros)
    2. Comprobaciones semánticas de todas las expresiones del programa, incluyendo la inicialización de variables, las expresiones asignadas a variables, operadores aplicados y condiciones de los controles de flujo.
        1. Se harán las comprobaciones de tipos y operaciones válidas para distinguir los diferentes tipos del lenguaje incluyendo, cuando sea necesaria, la conversión de tipos compatibles.
    3. Implementación a código utilizando ply.yacc
4. P3 - OPCIONAL Para poder **obtener la calificación máxima**, pero de manera **opcional**, se deberá elegir y desarrollar uno de los siguientes apartados (consultar la teoría para más detalle):
    1. Generación de código intermedio.
    2. Recuperación de errores.

Especificación del lenguaje

Lava es un pequeño lenguaje de programación imperativo y fuertemente tipado.

- El lenguaje trabaja con varios tipos de datos básicos.
- También permite definir tipos compuestos o registros.
- Las estructuras de control de flujo básicas que contiene son el salto y el bucle condicional.
- Permite la declaración y llamada de funciones.

```Java
record Vector(float x, float y);
record Planet(Vector position, Vector velocity, float mass, boolean active);

// Actualiza la posición utilizando la velocidad
void move(Planet p) {
  p.position.x = p.position.x + p.velocity.x;
  p.position.y = p.position.y + p.velocity.y;
}

/*
  Computa la energía cinética
  i.e. ½ * m * v²
*/
float energy(Planet p) {
  float squareSpeed = p.velocity.x * p.velocity.x + p.velocity.y * p.velocity.y;
  return 0.5 * p.mass * squareSpeed;
}

Planet earth = new Planet(new Vector(0, 0), new Vector(1, 2), 5, true);

// Simula 3 etapas
int step = 0;
while (step < 3) {
  if (!earth.active) {
    move(earth);
  }
  step = step + 1;
}

// ¿Es la energía suficiente?
if (energy(earth) > 10.0) {
  print('Y');
} else {
  print('N');
}
```

Comentarios

Se permiten comentarios, en cualquier parte del código, que deben ser tratados - e ignorados - por completo en el analizador léxico. Los tipos de comentarios son:

- **De una línea:** Comienzan por `//` y terminan con un salto de línea.
- **Multi-línea:** Comienzan por `/*` y terminan con `*/`. El contenido puede contener cualquier carácter, incluyendo el salto de línea.

Palabras reservadas

Las palabras reservadas del lenguaje tienen una condición especial y, por tanto, no pueden usarse como nombres de variables. Solo pueden usarse en minúsculas y son:

`true`	`false`	`int`	`float`
`char`	`boolean`	`void`	`return`
`if`	`else`	`do`	`while`
`break`	`print`	`new`	`record`

Variables y tipos

Todas las variables deberán ser tipadas en su declaración. A continuación, se detallan los tipos básicos que se manejan en `Lava`

Variable	Número entero	Número real	Carácter	Booleano
Tipo	`int`	`float`	`char`	`boolean`

Número entero

Declaración decimal; los caracteres posibles son [0-9]

- No se permiten ceros no significativos (por la izquierda)
- `int decimal = 495;`

La representación binaria comienza con `0b`; los caracteres posibles son [0-1]

- `int binary = 0b111101111;`

La declaración octal comienza con `0`; los caracteres posibles son [0-7]

- `int octal = 0757;`

La declaración hexadecimal comienza con `0x`; los caracteres posibles son [0-9A-F]

- `int hexadecimal = 0x1EF;`

Número real

Notación con punto decimal: se requiere parte entera, parte decimal y usa (`.`) como separador decimal

- `float dot_notation = 100.001;`

Notación científica (exponencial): utiliza `e` como separador de potencia de 10, positiva o negativa. La potencia deberá ser un número entero, pero el número base podrá ser tanto entero como decimal.

- `float scientific = 9.87e-2; // ~0.0987`
- `float scientific_pos = 5e5; // ~500000.0`

Carácter

Cualquier símbolo de la codificación ASCII-extendido, delimitado por comillas simples (`' '`)

- `char letter = 'a';`
- `char symbol = '¿';`

Booleano

Representan valores lógicos utilizando las palabras reservadas `true` y `false`

- `boolean active = true;`

Operaciones

Aritméticas

Binarias en notación infija (`+, -, *, /`) y unarias (`+, -`)

- `int suma = 10 + 5; // suma binaria`
- `float negación = -suma; // cambio de signo unario`

Booleanas

Lógicas binarias de conjunción (`&&`) y disyunción (`||`) y unaria de negación (`!`)

- `boolean and = true && false;`
- `boolean not = !true;`

Comparativas

Evalúan relación entre valores y devuelven un booleano (`>, >=, <, <=, ==`)

- `boolean equal = 5 == 5`
- `boolean greater = 10.3 > 200e-1`

Reglas de precedencia

Para la correcta interpretación de las expresiones que contienen múltiples operadores de diferente o igual tipo (aritméticas, booleanas y comparativas), se deben establecer reglas claras de precedencia y asociatividad.

En lugar de construir una gramática sintáctica donde la precedencia se codifique explícitamente a través de múltiples niveles de reglas de producción, se utilizará la funcionalidad precedence proporcionada por la librería **PLY**.

Esta herramienta simplifica significativamente el diseño del *parser* al permitir definir la precedencia y la asociatividad de los símbolos terminales (operadores) de manera declarativa. Se requiere que se establezcan:

1. **Niveles de precedencia:** El orden jerárquico en el que se deben evaluar los operadores (p. ej., la multiplicación tiene mayor precedencia que la suma).
2. **Asociatividad:** La dirección de evaluación para operadores del mismo nivel de precedencia (izquierda, derecha o no asociativa).

El conjunto de reglas de precedencia y asociatividad deberá abarcar todos los operadores del lenguaje `Lava`. Se sugiere seguir el estándar de precedencia de operadores de lenguajes como Java por su amplia adopción. Sin embargo, cualquier otro enfoque o variación es aceptable, siempre que la elección y el razonamiento detrás de los niveles de precedencia y asociatividad seleccionados se documenten y justifiquen detalladamente en la memoria técnica de la entrega.

Nota: Es crucial que los paréntesis mantengan el nivel más alto de precedencia, asegurando que las sub-expresiones encerradas se evalúen antes que las externas.

Variables de tipo registro

La especificación actual del lenguaje `Lava` define los registros como estructuras de datos compactas que integran tanto la definición del tipo como su inicialización.

```Java
/* Definición del registro (tipo) - record Nombre(tipo prop1, tipo prop2, ...)
Define la estructura de datos de forma inmutable y compacta. */
record Circle(float cx, float cy, float radius, char color);
record Square(float side, char color);

/* Instanciación: Se usa la palabra reservada 'new' para crear el objeto.
No se permite la declaración sin asignación inmediata en este formato */
Circle c = new Circle(0, 0, 10.2, 'R');
Square s = new Square(3.1e2, 'B');

// Acceso a propiedades
// Se utiliza el operador punto (.) para obtener los valores.
int circleArea = 3.1416 * c.radius * c.radius;
int squareArea = s.side * s.side;

// Operación de comparación: devuelve un booleano tras evaluar áreas.
boolean squareIsBigger = squareArea > circleArea;
```

Un registro se declara mediante la palabra reservada `record`, seguida de un identificador y una lista de parámetros tipados entre paréntesis que establecen sus propiedades inmutables.

Para crear una instancia, se utiliza el operador `new` pasando los argumentos correspondientes al constructor, mientras que el acceso a los datos se realiza de forma directa mediante la notación de punto - (`objeto.propiedad`)

Esta sintaxis permite integrar los registros de manera fluida en expresiones aritméticas y lógicas, facilitando el manejo de datos compuestos con una tipificación clara y estricta.

Componentes de la especificación

- Definición del tipo: La cabecera `record Nombre(campos)` actúa como el esquema estructural.
- Instanciación Obligatoria: La creación de la variable requiere el uso de `new` para asegurar que todas las propiedades tengan un valor desde el nacimiento del objeto. El acceso a propiedades antes de la instanciación no está permitido.
- Acceso Directo: El operador punto permite recuperar valores para ser usados como operandos en cálculos (ej. `c.radius`) o cualquier otro tipo de operación.
- Posibles tipos de los campos: Un campo de un registro podrá tener cualquier tipo básico o bien el de otro registro previamente definido.

```java
record Location(char city, char country);
record Price(float euro, float dollar, float yen);

// Registro con campos que a su vez son de tipo registro
record House(Location location, Price price);

// Instanciación anidada
House myHouse = new House(
  new Location('M', 'E'),
  new Price(100e100, 108e100, 16397e100));

float withTax = myHouse.price.euro * 1.1;
float euroToYen = myHouse.price.euro * 163.97;

// Llamada a la función de sistema 'print' para mostrar por consola
print(euroToYen == myHouse.price.yen);
```

Control de flujo

En cuanto al control de flujo, se definen dos estructuras de control: salto y bucle condicional.

Salto condicional

La especificación del lenguaje para el salto condicional se basa en una estructura de control de flujo que evalúa la veracidad de una condición lógica para ramificar la ejecución del programa.

- Bajo esta sintaxis, la palabra reservada `if` precede obligatoriamente a una expresión booleana encerrada entre paréntesis; si dicha expresión resulta verdadera, se acabaría ejecutando el bloque de sentencias delimitado por llaves (`{}`).
- En caso de que la condición sea falsa y exista una cláusula opcional `else`, el programa desviaría su flujo hacia el segundo bloque de sentencias delimitado, de nuevo, por llaves (`{}`).

Esta estructura permite integrar dentro de la condición una lógica compleja que combina operaciones aritméticas, relacionales y booleanas, garantizando que solo una de las dos rutas de ejecución sea procesada antes de reincorporarse a la secuencia normal del programa.

```java
if (f2 == 3 || b1 && b2 || 10 - 4 * i1 >= 0xFF - 1e-1) {
  f2 = f2 - 3;
} else {
  f2 = 10 - f1 * f1;
}
```

Bucle condicional

La especificación del lenguaje para el bucle condicional define una estructura de control iterativa que permite la ejecución repetida de un bloque de código mientras una condición booleana sea verdadera.

- La sentencia se inicia con la palabra reservada `while`, seguida de una expresión lógica entre paréntesis que se evalúa antes de cada iteración.
- Si el resultado es `true`, se ejecuta el cuerpo del bucle delimitado por llaves (`{}`).

Dentro de esta estructura, el lenguaje permite la alteración del flujo mediante la palabra reservada `break`, la cual provoca la salida inmediata del bucle, transfiriendo el control a la primera sentencia posterior al bloque del `while`.

Esta combinación permite gestionar tanto bucles controlados por una condición de entrada como terminaciones abruptas basadas en estados internos, garantizando flexibilidad en el procesamiento secuencial y la manipulación de variables de control o contadores.

```Java
int i;
int v;

do {
  if (v >= 10) {
    break;
  }

  v = (v + 1) * 2;
  i = i + 1;

  print(v);
} while (i < 4)
```

Funciones

La especificación actual del lenguaje para la definición de subrutinas o funciones adopta una sintaxis modular y estructurada.

- Una función se define especificando primero el tipo de dato que devuelve (puede ser `void`), seguido de su identificador y una lista de parámetros tipados entre paréntesis (puede ser vacía).
- El cuerpo de la función (conjunto de sentencias) se delimita obligatoriamente por llaves (`{}`) y debe contener al menos una sentencia de retorno mediante la palabra reservada `return` (excepto si el tipo de dato que devuelve es `void`, donde estaría prohibido incluirla)

El lenguaje soporta la sobrecarga de funciones, lo que permite definir múltiples funciones con el mismo nombre siempre que su firma (número o tipo de argumentos) sea distinta.
Al invocar una función, las expresiones pasadas como argumentos se evalúan y se asignan a los parámetros correspondientes, ejecutando el bloque de sentencias hasta alcanzar el valor de retorno, el cual puede ser integrado directamente en expresiones más complejas.

Este ejemplo muestra una función que calcula el promedio de dos números reales.

```Java
float calcularPromedio(float a, float b) {
    float suma = a + b;
    return suma / 2;
}

// Llamada a la función
float resultado = calcularPromedio(10.5, 5.5);
```

En este segundo caso, se utilizan registros o tipos complejos como parámetros de entrada de la función.

```Java
record Point(int x, int y);
record Line(Point a, Point b);

Line lineFromCoordinates(int x1, int y1, int x2, int y2) {
  return new Line(new Point(x1, y1), new Point(x2, y2));
}

Line l = lineFromCoordinates(0, 0, 4, 2);
```

Estructura del programa

Hay que tener en cuenta un serie de casos límite que deberán ser permitidos:

1. Un fichero vacío es válido.
2. Un fichero con solo saltos de línea es válido.
3. Un fichero con solo delimitadores (`;`) es válido.
4. Las sentencias pueden estar separadas por uno o muchos delimitadores (`;`)
5. Las declaraciones de funciones no llevan delimitador al final (`;`) ya que usan llaves (`{}`)
6. Un programa es un conjunto de sentencias y/o declaraciones de funciones.

```Java
;;
;
int test = -0b1;;;  ; ;

boolean check() { return true; }
;
;;;;;
```

Sentencias

1. Evaluación de expresiones, de cualquier tipo o combinación válida de operaciones aritméticas, booleanas o de comparación.
2. Declaración/Asignación de variables de los diferentes tipos.

Las expresiones pueden ser literales, operaciones, llamadas a funciones, accesos a propiedades de registros, etc. es decir, todo aquello que represente un valor.

Las declaraciones comienzan por el tipo de variable seguido del nombre de la variable. Las asignaciones comienzan por un nombre de variable, seguido del signo igual y terminan con una expresión.

No se permite la asignación de valores en mitad de una expresión: serán incorrectas las sentencias de tipo:

```Java
A +  4 * B = 3 - 7; // Error: asignación (B=3) en una expresión
```

Las variables pueden asignarse en el momento de la declaración. En caso de realizar la declaración y asignación por separado, la variable debe declararse antes de asignarse.

El lenguaje permite declarar (pero no declarar y asignar) una lista de variables separadas por comas, indicando que todas comparten el mismo tipo.

```Java
int a, b;
a = 10;
b = a * a - a;

float d, e = 0xFF / 0702; // Error sintáctico:
                          // No se permite la multideclaración con asignación
```

Se actualizará el documento para la entrega P3 con más información semántica sobre las sentencias.

Controles semánticos

Reglas de tipos

Se actualizará el documento para la entrega P3

Conversiones automáticas

Se actualizará el documento para la entrega P3

Diseño avanzado

Generación de código intermedio

Se actualizará el documento para la entrega P3

Recuperación de errores

Se actualizará el documento para la entrega P3

Entregas

Este apartado establece las normas generales aplicables a todas las fases del proyecto.

1. Cada entrega deberá **realizarse estrictamente en parejas** constituidas de forma permanente, no permitiéndose el cambio de integrantes entre las distintas fases.
2. El archivo de entrega debe seguir **obligatoriamente el formato de nomenclatura** detallado a continuación, sustituyendo `X` por el número de la práctica y los nombres y apellidos por los de los autores.
   - `PX_NOMBRE1_APELLIDO1_NOMBRE2_APELLIDO2.zip`
3. Es fundamental asegurar que el código no genere `WARNINGS` o `ERRORS` procedentes de la librería `ply`; cualquier aviso del sistema deberá ser eliminado o, en casos excepcionales, debidamente justificado en la memoria.
4. La **originalidad** es un requisito indispensable: cualquier indicio de plagio o copia entre grupos resultará en el suspenso de la práctica.
5. Respecto al uso de **Inteligencia Artificial,** se considera una herramienta de apoyo y no una sustitución del trabajo del alumno; su uso abusivo o el desconocimiento del código generado por esta podrá ser penalizado. El equipo docente se reserva el derecho de convocar defensas orales para verificar la autoría y el conocimiento profundo del material entregado.

Los criterios de evaluación serán:

- **Funcionalidad:** El sistema debe devolver la salida esperada sin errores. Se recomienda adjuntar una batería de pruebas propias para validar el comportamiento.
- **Dominio de `ply`:** Aplicación correcta de las herramientas trabajadas en los laboratorios y manuales oficiales de la librería.
- **Memoria técnica:** Exposición clara y justificada de las decisiones tomadas. Debe ser un reflejo fiel que guíe al corrector a través del código.

⚠️ ***El incumplimiento del formato de entrega o de los nombres de archivos indicados en cada entrega supondrá una pérdida de puntos automática.***

Analizador léxico P1

Para la primera entrega del proyecto, se requiere el desarrollo del analizador léxico utilizando la librería `ply.lex`. Esta fase inicial debe sentar las bases de la gramática del lenguaje mediante el reconocimiento de los tokens que más tarde serán los símbolos terminales de la misma, por lo que el código entregado debe ser lo más sólido y definitivo posible, permitiendo sólo ajustes menores en entregas posteriores basados en el feedback recibido.

La entrega debe estar compuesta obligatoriamente por dos archivos: `lexer.py`, que contendrá el analizador léxico, y `main.py`, que servirá como punto de entrada.

- Al ejecutar el comando `main.py <file>`, el programa debe procesar el archivo de entrada y generar un archivo de salida con el mismo nombre y extensión `.token`.
- El formato de salida debe ser estrictamente de un token por línea, siguiendo la estructura
  - `{ TIPO, VALOR, LÍNEA, COLUMNA-INICIO, COLUMNA-FIN }`

```shell
// Fichero de entrada - example.lava
int a = 28;
float b = 24.9;

// Ejecución
> python main.py example.lava

// Fichero de salida - example.token
// { TIPO, VALOR, LÍNEA, COLUMNA-INICIO, COLUMNA-FIN }
INT, int, 1, 0, 3
ID, a, 1, 4, 5
ASSIGN, =, 1, 6, 7
INT_VALUE, 28, 1, 8, 10
SEMICOLON, ;, 1, 10, 11
FLOAT, float, 2, 0, 5
ID, b, 2, 6, 7
ASSIGN, =, 2, 8, 9
FLOAT_VALUE, 24.9, 2, 10, 14
SEMICOLON, ;, 2, 14, 15
```

Memoria

Junto con el código, se debe adjuntar una memoria detallada, en formato PDF, que documente las decisiones de diseño del analizador, profundizando en el razonamiento y justificación de cada una de ellas.

- Elección de tokens: Listado de identificadores, palabras reservadas y símbolos, justificando su clasificación y procesamiento.
- Gestión de caracteres: Especificación de los caracteres que se ignoran (como espacios en blanco o tabulaciones) y la lógica para el rastreo de líneas y columnas.
- Tratamiento de comentarios: Explicación de las expresiones regulares utilizadas para omitir comentarios de una o varias líneas sin interrumpir el flujo del análisis.

Analizador sintáctico P2

Para la segunda entrega del proyecto, se requiere el desarrollo del analizador sintáctico utilizando la librería `ply.yacc`. Esta fase intermedia debe consolidar la definición de la gramática que generará el lenguaje especificado en este enunciado. Está gramática será más tarde expandida con los controles semánticos, por lo que el código entregado debe ser lo más sólido y definitivo posible, permitiendo sólo ajustes menores en entregas posteriores basados en el feedback recibido.

La entrega debe estar compuesta obligatoriamente por dos archivos: `lexer.py` (el mismo que en la primera entrega, con las mínimas correcciones indicadas en el feedback), `parser.py`, que contendrá el analizador sintáctico, y `main.py`, que servirá como punto de entrada.

- Al ejecutar el comando `main.py <file>`, el programa debe procesar el archivo de entrada y no mostrar ninguna salida si el fichero es una palabra perteneciente al lenguaje que genera la gramática. En caso de que el fichero sea incorrecto, deberá mostrar los errores correspondientes.
- El analizador léxico no deberá ejecutarse "manualmente", es decir, se deberá referenciar al analizador léxico en la definición del analizador sintáctico, para que este último lo utilice durante el proceso de análisis, tal y como se ha explicado en clase.
- Opcionalmente, se introducirá una opción con un comando estilo `main.py –token <file>` cuyo comportamiento será solo ejecutar el analizador léxico y exportar el fichero `.token` siguiendo las reglas de la entrega anterior.

Es muy importante que no se encuentren conflictos a la hora de generar las tablas LALR. Estos conflictos deben solucionarse con modificaciones en la gramática o el uso correcto de la herramienta de precedencia. Cualquier conflicto no controlado, bajará la nota drásticamente.

```Shell
// Fichero de entrada - example.lava
int a = 28;
float b = 24.9;

// Ejecución
> python main.py example.lava
Generating LALR tables...

--------------------------------------------------------------------------------

// Fichero con errores - error.lava
int a float = 3.3;

// Ejecución
> python main.py example.lava
Generating LALR tables...
[ERROR] Token 'FLOAT' inesperado en la línea 1
```

Memoria

Junto con el código, se debe adjuntar una memoria detallada, en formato PDF, que documente las decisiones de diseño del analizador, profundizando en el razonamiento y justificación de cada una de ellas.

- Tokens: Repaso de los tokens (símbolos terminales) capturados por el analizador léxico.
- Definición formal de la gramática: Una gramática formal es un sistema definido como una 4-tupla, utilizado para describir lenguajes formales mediante reglas de producción que generan cadenas válidas de símbolos terminales (alfabeto) a partir de un símbolo inicial (axioma) y símbolos no terminales: $G = (V_n, V_t, P, S)$
- Desarrollo/Diseño de la gramática: Cuál es la estructura principal del lenguaje, que se permite y qué no, cómo se ha definido esto en el diseño, como funcionan las expresiones, etc.

Análisis semántico y partes opcionales P3 P3 - OPCIONAL

Se actualizará el documento para la entrega correspondiente.